

CS 223: Systems Programming

Instructor: Kelly Shiptoski

Lecture: 2, Week: 1

Agenda

Admin recap

Working with relative / absolute paths review

Stack diagrams

I/O

Arrays

Strings

Structs

Course Format

Lectures

- Combination of slides + live coding / debugging

Labs

- Code practice + ask me any questions
- Worksheets (completed in pairs)
- Timed quizzes (individual)

Assignments

- Weekly (due on Fridays)

Exams

- Midterm exam
- Final exam

Course Topics

- C programming language fundamentals.
- Memory management.
- Debugging tools (gdb, valgrind, etc).
- Data representations “under-the-hood”.
- Working with UNIX, git, and terminal editors.
- **Core operating systems concepts.**
- **Concurrent programming (processes and threads).**
- Performance tradeoffs.
- Introduction to the **Rust** programming language.

Grade Breakdown

Final Exam: 35%

Midterm Exam: 35%

Homework: 10%

Code Challenges / Quizzes: 15%

Lab Practice / Class Participation: 5%

Java vs C

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println(x: "Hello World!");  
    }  
}
```

```
#include <stdio.h>  
  
int main(int argc, char **argv) {  
    printf("Hello World!\n");  
    return 0;  
}
```

Java vs C

To compile:

```
gcc hello.c -o hello
```

```
javac Hello.java
```

To run:

```
./hello
```

```
java Hello
```

Exercise

Visualize the directory hierarchy after running the following commands:

```
> pwd
/home/kelly
> mkdir one
> cd one
> mkdir two
> touch talk.c
> cd ..
> touch listen.c
> cd
> touch sing.c
```

The Stack

Execution stack: keeps track of active functions.

We will use stack diagrams to visualize program flow.

For checking your work: <https://pythontutor.com/c.html>.

Stack Frames

A stack frame is created for each function.

- Contains parameters, local variables.
- Topmost frame is currently executing function.
- Frame is pushed to stack when function is called.
- Frame is popped off stack when it returns.

Java → C

What is the output of this program?

```
#include "stdio.h"

int func(int a, int b) {
    a += 5;

    if (a > 10) {
        return 0;
    } else {
        return a - b;
    }
}

int main() {
    int x, y;
    x = 4;
    y = 7;

    for (int i = 0; i < 3; i++) {
        y = func(a: x, b: y);
        printf("%d, %d\n", x, y);
    }
}
```

Print Functions

C: `printf()`

Java: `System.out.printf()`

```
printf("%d %s\t%f\n", 6, "hello", 3.4);
```

```
System.out.printf(format: "%d %s\t%f\n", ... args: 6, "hello", 3.4);
```

Both functions output: `6 hello 3.400000`

Input

`scanf()` : for reading in values of different types.

Uses format string like `printf()`.

The arguments are read into an array.

The arguments are the memory locations of the stored values.

scanf

The arguments are read into an array.

The arguments are the memory locations of the stored values.

```
char s[100];  
int x;  
float y;  
  
scanf("%s %d %f", s, &x, &y);
```

s is the base address of the string array.

&x is the **address of the variable x** in memory.

&y is the **address of the variable y** in memory.

Example

```
int main() {  
    printf("Do you like jokes? (y/n): ");  
  
    char response = 0;  
    scanf("%c", &response);  
  
    printf("Your response: %c\n", response);  
  
    if (response == 'y') {  
        printf("Me too!\n");  
    } else if (response == 'n') {  
        printf("Me neither!\n");  
    } else {  
        printf("Are directions difficult for you?\n");  
    }  
  
    return 0;  
}
```

Booleans

There are **no booleans** in C.

0 is always **false**.

Every other value is **true**.

Helpful to treat **0 as false** and **1 as true**.

Arrays: Java vs C

```
class Array {
    public static void main(String[] args) {
        int i, size = 0;
        int[] arr = new int[10];

        for (i = 0; i < 10; i++) {
            arr[i] = i;
            size++;
        }

        arr[3] = 100;

        System.out.printf(format: "array of %d items:\n", size);

        for (i = 0; i < 10; i++) {
            System.out.printf(format: "%d\n", arr[i]);
        }
    }
}
```

```
int main() {
    int i, size = 0;
    int arr[10];

    for (i = 0; i < 10; i++) {
        arr[i] = i;
        size++;
    }

    arr[3] = 100;
    printf("array of %d items:\n", size);

    for (i = 0; i < 10; i++) {
        printf("%d\n", arr[i]);
    }

    return 0;
}
```

Arrays in C

In C, the size of an array is set at compile time and **cannot change during runtime**.

Other languages, like Java, provide both **statically** and **dynamically** allocated arrays.

- Statically allocated: `Java.util.Arrays`
- Dynamically allocated: `Java.util.ArrayList`

Arrays in C

There is **no bounds checking** in C.

It is very easy to write/read elements out of bounds of an array.

You may (or may not) notice the problem, depending on what memory you access...

Out of Bounds Access: C

```
int main() {  
    int bad[20] = {[0]=1, [1]=2, [2]=3};  
  
    printf("bad[2] = %d\n", bad[2]);  
    printf("bad[20] = %d\n", bad[20]);  
    return 0;  
}
```

```
> ./out-of-bounds  
bad[2] = 3  
bad[20] = -1195769735
```



Out of Bounds Access: Java

```
class BadArray {  
    public static void main(String[] args) {  
        int[] bad = { 1, 2, 3, 4 };  
  
        System.out.printf(format: "bad[2] = %d\n", bad[2]);  
        System.out.printf(format: "bad[4] = %d\n", bad[4]);  
    }  
}
```

```
bad[2] = 3
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 4 out of bounds for length 4
```

Exercise: Fibonacci

Write a C program that computes the first 10 Fibonacci numbers and stores them in an array.

Strings in C

```
// Array of chars  
char greeting[5] = "ciao";  
  
// Pointer to read-only array of chars.  
const char *message = "This is the breakdown";
```

Strings in C

In C, a **string** is an **array of chars**.

Not every **array of chars** is a **string**, but every **string** is an **array of chars**.

Strings in C

Strings in C **must** be large enough to hold the **number of characters + 1**.

Last character is a **null character** that denotes the end of the string (`'\0'`).

C strings are **null-terminated**.

String Library in C

```
#include <string.h>
```

strlen	Length of string
strcpy	Copy
strcat	Concatenate
strcmp	Compare

Let's do some examples.

```
#include <stdio.h>
#include <string.h>

int main() {
    char greeting[5] = "ciao";
    const char *message = "This is the breakdown";

    char buffer[10];
    buffer[0] = 'h';
    buffer[1] = 'i';
    buffer[2] = '\0';

    printf("%s\n", greeting);
    printf("%s\n", message);
    printf("%s\n", buffer);

    int len = strlen(message);
    printf("Len of message is: %d\n", len);

    strcpy(buffer, "test");
    printf("Buffer is: %s\n", buffer);

    return 0;
}
```

```
ciao
This is the breakdown
hi
Len of message is: 21
Buffer is: test
```

strlen

```
const char *message = "Hi there";  
  
int len = strlen(message);  
printf("Len of message is: %d\n", len);
```

```
Len of message is: 8
```

`strlen` returns the number of characters in the string, not including the null terminator.

strcpy

```
char buffer[10];  
buffer[0] = 'h';  
buffer[1] = 'i';  
buffer[2] = '\\0';  
  
strcpy(buffer, "there");  
printf("Buffer is: %s\\n", buffer);
```

```
Buffer is: there
```

strcpy copies data from one string into the memory of another string.

strcat

```
char buffer[10];  
buffer[0] = 'h';  
buffer[1] = 'i';  
buffer[2] = '\\0';  
  
strcat(buffer, "there");  
printf("Buffer is: %s\\n", buffer);
```

```
Buffer is: hithere
```

`strcat` joins two strings by concatenating a source string to the end of destination string.

Structs in C

No classes in C.

To bundle data together, we use a **struct**.

In Java, classes hold data **and** define methods.

In C, **structs** hold data and **functions** define methods.

Example

```
struct studentT {
    char name[64];
    int age;
    float gpa;
    int grad_yr;
};

int main() {
    struct studentT student1, student2;

    strcpy(student1.name, "Duncan Idaho");
    student1.age = 18 + 2;
    student1.gpa = 3.5;
    student1.grad_yr = 2020;

    // %s - string
    // %d - int
    // %g - float
    printf("name: %s, age: %d, gpa: %g, year: %d\n", student1.name, student1.age,
        student1.gpa, student1.grad_yr);

    return 0;
}
```

Typedef for C Structs

```
typedef struct studentT {
    char name[64];
    int age;
    float gpa;
    int grad_yr;
} studentT;

int main() {
    studentT student1, student2;

    strcpy(student1.name, "Duncan Idaho");
    student1.age = 18 + 2;
    student1.gpa = 3.5;
    student1.grad_yr = 2020;

    // %s - string
    // %d - int
    // %g - float
    printf("name: %s, age: %d, gpa: %g, year: %d\n", student1.name, student1.age,
        student1.gpa, student1.grad_yr);

    return 0;
}
```

Arrays and Structs in Memory

```
struct dataT {
    int v[3];
    char c;
    float y;
};

int main() {
    int arr[3];
    arr[0] = -1;
    arr[1] = 5;
    arr[2] = 3;

    struct dataT data;
    data.v[0] = 2;
    data.v[1] = -3;
    data.v[2] = 0;
    data.c = 'g';
    data.y = 3.14;

    for (int i = 0; i < 3; i++) {
        printf("%d\n", data.v[i]);
    }

    return 0;
}
```

Stack Diagram Examples

On board

Difference between high and low level languages